Integração Front-end e Back-end com C#

AULA 15

O fluxo do dado no mundo real — do formulário ao banco de dados. Nesta aula, vamos percorrer cada etapa da jornada de uma informação, desde o momento em que o usuário preenche um campo até o instante em que o servidor processa, valida e persiste os dados com segurança.



O Ciclo de Vida da Informação

Toda interação em uma aplicação web segue um fluxo bem definido. Compreender esse ciclo é fundamental para construir sistemas robustos e confiáveis.



A Interface como Porta de Entrada

O usuário interage com formulários, botões e campos — tudo que ele vê e toca no navegador.



O Servidor como Guardião

O back-end recebe, valida e processa os dados antes de qualquer ação ser executada.



A Persistência Segura

Os dados validados são armazenados com integridade, prontos para serem recuperados quando necessário.

A ponte entre esses mundos são as **requisições HTTP** — principalmente os verbos **GET** (leitura) e **POST** (envio de dados).

O Front-end: Coletando o que importa

Formulários HTML

Os formulários são a interface primária de comunicação entre o usuário e o sistema. Cada campo — texto, e-mail, senha, seleção — representa um dado que será enviado ao servidor.

- Tags `<form>` com atributos `action` e `method`
- Campos com `name` para identificação no back-end
- Botões de envio com `type="submit"`

Captura e Envio de Dados

Com JavaScript, é possível interceptar o envio, capturar os valores e transmiti-los ao servidor em formatos estruturados como **JSON** ou **Form Data**.

Validação Client-Side

A primeira barreira de defesa: verificações no navegador melhoram a experiência do usuário, mas **nunca substituem** a validação no servidor.

- Campos obrigatórios preenchidos
- Formato de e-mail válido
- Senhas com tamanho mínimo

A Chegada ao Back-end

Quando o dado cruza a rede e chega ao servidor, o **ASP.NET MVC** entra em ação com mecanismos elegantes para receber e mapear automaticamente as informações.

1

Controller: O Controlador

O **Controller** é o ponto de entrada das requisições. Ele decide qual ação executar, recebe os dados e devolve a resposta adequada — seja uma View ou um JSON.

2

Rotas e Verbos HTTP

As **rotas** definem qual URL aciona qual método. Os verbos **GET**, **POST**, **PUT** e **DELETE** indicam a intenção da operação e ajudam a organizar a API de forma semântica.

3

Model Binding Automático

O C# mapeia automaticamente os campos do formulário para as propriedades do seu modelo. Basta nomear os campos igual às propriedades e o ASP.NET faz o trabalho pesado.

Validação: Onde o Erro Morre

Data Annotations no C#

O .NET oferece atributos poderosos que você aplica diretamente nas propriedades do modelo para definir regras de validação de forma declarativa:

`[Required]`

Campo obrigatório — não pode ser nulo ou vazio.

`[StringLength(50)]`

Limita o tamanho máximo (e mínimo) de textos.

`[EmailAddress]`

Valida automaticamente o formato de e-mail.

`[Range(0, 120)]`

Restringe valores numéricos a um intervalo seguro.

Regra de Ouro

Nunca confie nos dados do cliente. A validação no front-end é uma cortesia ao usuário — a validação no servidor é uma obrigação de segurança.

Use `ModelState.IsValid` no controller para verificar todas as regras antes de processar qualquer dado.

⚠ Retorne sempre **feedback claro** ao usuário indicando exatamente qual campo possui erro e por quê.

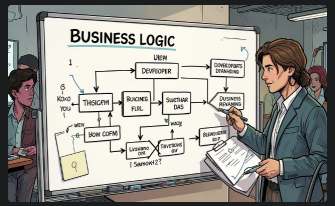
Processamento no Servidor

Com os dados validados, o servidor executa a lógica de negócio — o coração da aplicação. É aqui que as regras reais acontecem.



Transformação de Dados

Antes de usar os dados, o servidor os limpa e formata: trim em strings, conversão de tipos, normalização de e-mails e datas. Dados brutos nunca são processados diretamente.



Lógica de Negócio

Regras como verificação de duplicidade, cálculo de valores, permissões de acesso e fluxos condicionais são executadas aqui — isoladas da camada de interface e de dados.



Integração com Dados

Com o Entity Framework, salvar os dados no banco é seguro e elegante. O ORM gera as queries automaticamente, prevenindo injeções de SQL e garantindo transações consistentes.



O Poder do ASP.NET Core

O ASP.NET Core é a plataforma de escolha para aplicações web modernas em C#. Sua arquitetura foi desenhada para performance, segurança e escalabilidade desde o início.

Alta Performance

Um dos frameworks web mais rápidos do mercado, com pipeline otimizado e suporte nativo a async/await.

Segurança Integrada

Proteções contra CSRF, XSS e outros ataques comuns já vêm configuradas por padrão no framework.

Entity Framework

ORM nativo do .NET que conecta seu modelo C# diretamente ao banco de dados com mapeamento automático e migrations.

Escalabilidade

Arquitetura modular e suporte a containers Docker permitem escalar horizontalmente conforme a demanda cresce.

Estudo de Caso: Cadastro de Usuário

Vamos percorrer o fluxo completo de um cadastro — do formulário à persistência — incluindo o tratamento de erros e o uso de logs para diagnóstico.



O diagrama acima ilustra o caminho completo percorrido pelos dados de um formulário de cadastro, desde a interface do usuário até a persistência no banco de dados, passando por validação e lógica de negócio.

Tratamento de Exceções

Use blocos `try-catch` para capturar falhas inesperadas. Nunca exponha detalhes técnicos ao usuário — mostre mensagens amigáveis e registre o erro internamente.

Logs para Rastreamento

Utilize o `ILogger` do ASP.NET Core para registrar tentativas de submissão, validações falhas e exceções. Logs são essenciais para diagnosticar problemas em produção sem depender de relatos do usuário.

Checklist de Segurança

Dados vindos do usuário são potencialmente maliciosos. Cada ponto de entrada é uma vulnerabilidade em potencial — e sua aplicação precisa estar preparada.

Prevenção contra SQL Injection

Nunca concatene strings para montar queries SQL. Use sempre o **Entity Framework** ou parâmetros tipados. O ORM escapa automaticamente os valores, tornando injeções de SQL impossíveis.

Proteção contra XSS (Cross-Site Scripting)

O Razor do ASP.NET já faz **encoding automático** de saídas HTML. Ao exibir dados do usuário na tela, certifique-se de não usar `Html.Raw()` em conteúdo não confiável — isso desativa a proteção.

Sanitização Rigorosa de Entrada

Valide **tipo, tamanho e formato** de cada campo antes de processar. Rejeite tudo que não estiver dentro dos parâmetros esperados. Regra prática: *whitelist* é sempre mais seguro que *blacklist*.

Regra fundamental: Nunca processe dados brutos sem filtro. Cada dado que entra no seu sistema deve passar por validação, sanitização e verificação de integridade antes de qualquer uso.

Próximos Passos

A integração front-end/back-end é a base de toda aplicação web séria. Agora é hora de praticar e consolidar o conhecimento.

🎯 Desafio Prático

Implemente uma **validação customizada** em C# criando um atributo que herde de `ValidationAttribute`. Teste com cenários como CPF, CEP ou senhas com requisitos específicos.

🔬 Integração Contínua

Use o **Postman** para testar seus endpoints diretamente, sem depender do front-end. Simule requisições válidas e inválidas para garantir que seu controller responde corretamente em todos os cenários.

✅ Conclusão da Aula

O sucesso de uma aplicação depende da **integridade dos dados** que chegam ao servidor. Validação, sanitização e processamento seguro não são opcionais — são a fundação de qualquer sistema confiável.

> "Dados não validados são a principal porta de entrada para vulnerabilidades. Trate cada requisição como potencialmente hostil até provar o contrário."